

Subject: CS336 Assignment 1, Tokenizer Class

Date: from 2026/7/20 to 2026/7/21

Contents

Complete Tokenizer Class	1
---------------------------------	----------

Complete Tokenizer Class

Encode: Merge Order (Rank)

BPE 的编码过程必须严格重现训练时的“演化历史”。在训练阶段，每一对合并规则产生的先后顺序都被记录了下来，这个顺序就是 **Rank**。

- **Rank 0**: 最早被合并的规则（通常是最高频的字节组合）；
- **Rank N**: 最后被合并的规则。

铁律. 编码时优先合并 Rank 最小的字节对，与它在序列中出现的先后位置无关。

Encode Walk-through: target

场景设定：输入单词 **target**，初始字节序列为 [t, a, r, g, e, t]。假设训练好的 merges 规则按 Rank 从小到大排序如下：

Rank 0: e + t → et Rank 1: a + r → ar
Rank 2: g + et → get Rank 3: t + ar → tar

轮次	当前序列	命中规则的候选对	决策与结果
Round 1	[t, a, r, g, e, t]	(e, t) R0; (a, r) R1	选 R0 → et
Round 2	[t, a, r, g, et]	(a, r) R1; (g, et) R2	R1 < R2 → ar
Round 3	[t, ar, g, et]	(g, et) R2; (t, ar) R3	R2 < R3 → get
Round 4	[t, ar, get]	(t, ar) R3	唯一候选 → tar

FINAL RESULT: [tar, get]

Encode: Implementation

编码过程可以概括为一个不断重复的五步循环：

```
while True:
    1. 找出当前序列中所有相邻的 pair
    2. 检查这些 pair 是否在 merges 规则中
    3. 如果都不在，停止循环
    4. 如果有多个在，找出 rank 最小的那个 best_pair
    5. 执行合并：将序列中所有的 best_pair 替换为新 Token
```

Decode

解码是编码的逆过程，分三步完成：

1. **查表**. 遍历 ID 列表，从 vocab 中找到对应的 bytes；
2. **拼接**. 把所有 bytes 拼成一个长字节串；
3. **解码**. 调用.decode('utf-8') 把字节串还原为文本。

Decode: Invalid Byte Streams

致命问题. 由于 BPE 是基于统计的, 模型有时可能会生成 (或者被截断) “半个汉字”.

- 汉字 “牛” 的 UTF-8 编码是 `b'\xe7\x89\x9b'` (3 个字节);
- 场景: 模型只生成了前两个字节 `b'\xe7\x89'`, 随后就被截断 (例如达到 `max_len`);
- 后果: 直接 `decode('utf-8')` 会抛出 `UnicodeDecodeError`, 导致整个服务崩溃.

解决方法. 在解码时使用 `errors='replace'`:

```
text = byte_stream.decode('utf-8', errors='replace')
```

一旦检测到不符合 UTF-8 编码规则的非法字节, 就用 U+FFFD (`\ufffd`, Unicode 替换字符) 替换该字节, 而不是中断整个解码过程, 然后继续解码后续字节.